

EXPERIMENT – 5

Name: Aryan	UID: 24BAI70179
Section/Group: 24AIT-NTPP 2A	Branch/Semester: CSE_AIML (CS221) / 5th
Subject Code: 24 CSP-337	Subject Name: Full Stack - II
Date of Performance:	Submitted to: Ms. Lovleen Kaur (E20454)

Experiment – 5.1

1. AIM

To design and implement a complete REST API for User resource management using Spring Boot, Spring Data JPA and an in-memory H2 database, and to consume the API from a simple web frontend.

2. OBJECTIVE

1. To understand the structure of a Spring Boot Maven project and the role of each layer (model, repository, service, controller, DTO).
2. To implement Create, Read, Update and Delete (CRUD) operations as REST endpoints.
3. To persist User data using Spring Data JPA and the H2 in-memory database.
4. To validate incoming request bodies using Bean Validation annotations.
5. To consume the REST API from a browser-based frontend using the Fetch API.

3. SOFTWARE REQUIREMENT

1. Java JDK 17 or above.
2. Apache Maven (or Maven Wrapper bundled with the project).
3. Visual Studio Code with Extension Pack for Java.
4. Google Chrome or any modern web browser.
5. Live Server extension (for serving the frontend).
6. Spring Boot 3.4.x with Spring Web, Spring Data JPA, Validation and H2.

4. THEORY

REST (Representational State Transfer) is an architectural style in which resources are identified by URIs and manipulated with standard HTTP methods. In this experiment the resource is a User, exposed at /api/users.

Spring Boot reduces configuration by providing auto-configuration and an embedded Tomcat server. A typical REST application is split into layers: the Controller receives HTTP requests, the Service contains business logic, the Repository talks to the database through Spring Data JPA, and the Model is a JPA entity mapped to a table.

A DTO (Data Transfer Object) such as UserRequest is used so that the client does not send the persistence entity directly. Bean Validation annotations (@NotBlank, @Size, @Email) are evaluated when the controller parameter is marked with @Valid.

H2 is an in-memory relational database. Data lives only while the application is running, which is suitable for laboratory work. Hibernate (the JPA provider) generates the schema from the entity when spring.jpa.hibernate.ddl-auto is set to update or create.

CORS (Cross-Origin Resource Sharing) must be enabled so that a frontend opened from another origin (for example Live Server on port 5500) can call the API on port 8080.

5. IMPLEMENTATION/PROCEDURE

1. Create a Spring Boot Maven project with dependencies Spring Web, Spring Data JPA, Validation and H2 Database.
2. Define the User JPA entity with fields id, uid, name and email, mapped to table users.
3. Create UserRepository extending JpaRepository<User, Long> to obtain built-in CRUD methods.
4. Create UserRequest DTO with validation annotations and ApiResponse wrapper for consistent JSON responses.
5. Implement UserService with create, getAll, getById, update and delete methods.
6. Expose REST endpoints in UserController: POST /api/users, GET /api/users, GET /api/users/{id}, PUT /api/users/{id}, DELETE /api/users/{id}.
7. Configure application.properties for H2, JPA and server port 8080. Enable CORS.
8. Build a frontend (HTML, CSS, JavaScript) that calls the API using fetch().
9. Run Experiment5Application from VS Code (or ./mvnw spring-boot:run) and open the frontend with Live Server.
10. Test all CRUD operations from the UI and optionally from Postman.

6. CODE

Representative backend classes used in Experiment 5.1 are shown below. The complete Maven project is submitted separately as a zip.

User entity (simplified):

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String uid;
```

```
private String name;  
private String email;  
// constructors, getters and setters  
}
```

UserController (CRUD mappings):

```
@RestController  
@RequestMapping("/api/users")  
public class UserController {  
    private final UserService userService;  
  
    @PostMapping  
    public ResponseEntity<?> create(@Valid @RequestBody UserRequest req) { ... }  
  
    @GetMapping  
    public ResponseEntity<?> getAll() { ... }  
  
    @GetMapping("/{id}")  
    public ResponseEntity<?> getById(@PathVariable Long id) { ... }  
  
    @PutMapping("/{id}")  
    public ResponseEntity<?> update(@PathVariable Long id,  
                                    @Valid @RequestBody UserRequest req) { ... }  
  
    @DeleteMapping("/{id}")  
    public ResponseEntity<?> delete(@PathVariable Long id) { ... }  
}
```

7. OUTPUT

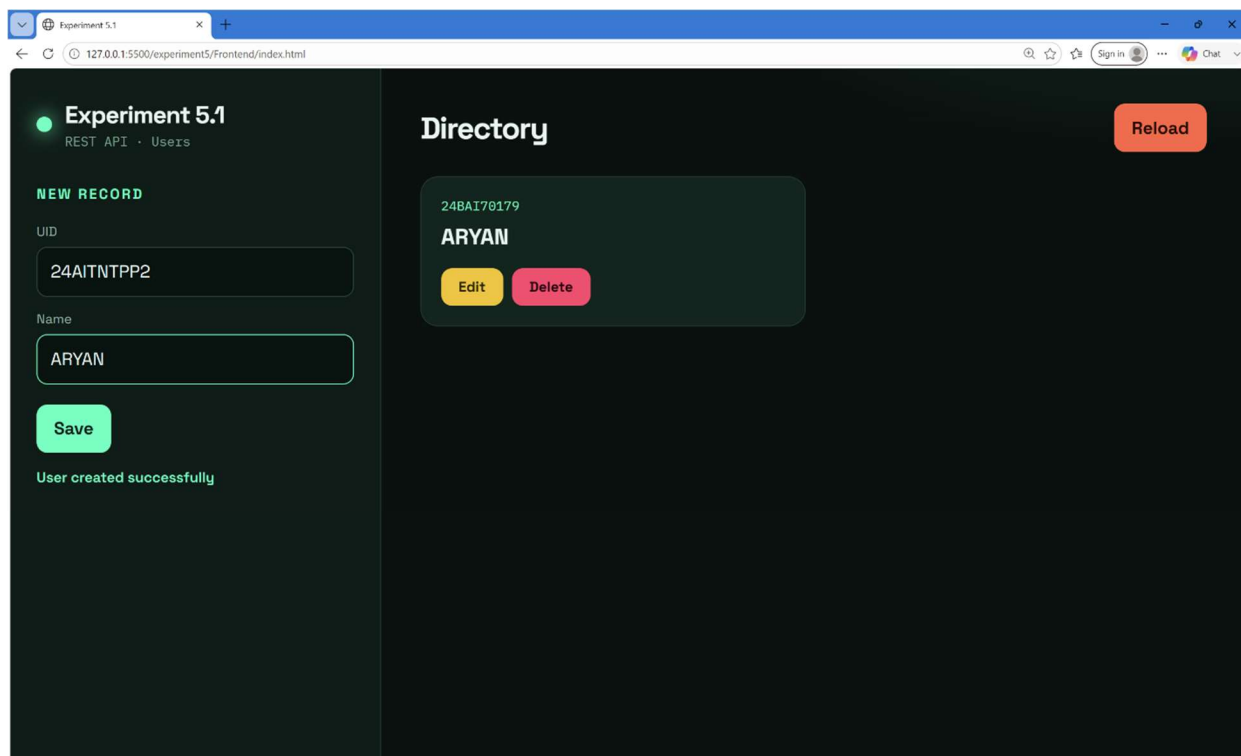


Figure 1: Output of Experiment 5.1

8. LEARNING OUTCOME

- Understood how a Spring Boot REST API is layered into controller, service, repository and entity.
- Implemented complete CRUD operations using HTTP methods and JSON request/response bodies.
- Used JPA and H2 to persist data without writing SQL by hand.
- Applied Bean Validation and consumed the API from a JavaScript frontend.

Experiment – 5.2

1. AIM

To enhance the REST API developed in Experiment 5.1 by adding centralized exception handling and request logging with a correlation identifier, so that errors are returned in a consistent format and every request can be traced in the server logs.

2. OBJECTIVE

1. To define custom exceptions such as `UserNotFoundException` for domain errors.
2. To implement a `@RestControllerAdvice` class (`GlobalExceptionHandler`) that maps exceptions to proper HTTP status codes and JSON bodies.
3. To handle `MethodArgumentNotValidException` and return field-wise validation errors.
4. To implement a servlet Filter that generates a `correlationId`, stores it in MDC, and logs method, URI and status.
5. To configure the logging pattern so that `correlationId` appears in every log line.

3. SOFTWARE REQUIREMENT

1. Same toolchain as Experiment 5.1 (JDK, Maven Wrapper, VS Code, Chrome).
2. Spring Boot 3.4.x with Spring Web, Validation, Data JPA and H2.
3. SLF4J / Logback (bundled with Spring Boot) and MDC support.
4. Postman (optional) to trigger error responses deliberately.

4. THEORY

When a REST API throws an exception from a controller or service, the default Spring Boot behaviour is to return a generic error page or a basic JSON problem body. For a laboratory-grade API we want a single, predictable envelope (success flag, message, data, timestamp) for both success and failure.

`@RestControllerAdvice` is a stereotype that intercepts exceptions thrown from any `@RestController`. Methods annotated with `@ExceptionHandler(ExceptionType.class)` convert that exception into a `ResponseEntity`. Typical mappings are: `UserNotFoundException` → 404, `MethodArgumentNotValidException` → 400, generic `Exception` → 500.

A servlet Filter sits in front of the `DispatcherServlet`. In this experiment the filter creates a UUID `correlationId`, puts it into SLF4J MDC, logs the incoming method and URI, continues the chain, then logs the response status and removes the MDC key. The logging pattern in `application.properties` includes `%X{correlationId}` so every line printed during that request carries the same id. This is the foundation of distributed tracing.

5. IMPLEMENTATION/PROCEDURE

1. Start from the working Experiment 5.1 project.
2. Add `UserNotFoundException` as an unchecked exception thrown when a user id does not exist.
3. Add `GlobalExceptionHandler` with handlers for `UserNotFoundException`, `MethodArgumentNotValidException` and `Exception`.
4. Implement `LoggingFilter`: generate `correlationId`, `MDC.put`, log request, `chain.doFilter`, log status, `MDC.clear`.
5. Register the filter for `/api/*` (`FilterRegistrationBean` or `@Component + @Order`).
6. Update `logging.pattern.console` to include `[%X{correlationId}]`.
7. Keep CORS enabled so the frontend continues to work.
8. Run the application and test: valid CRUD, unknown id (404), invalid body (400). Observe `correlationId` in the console.

6. CODE

GlobalExceptionHandler (core idea):

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ApiResponse<?>> handleNotFound(UserNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(ApiResponse.error(ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiResponse<?>> handleValidation(
        MethodArgumentNotValidException ex) {
        // collect field errors and return HTTP 400
    }
}
```

LoggingFilter (core idea):

```
@Component
public class LoggingFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest req,
        HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String cid = UUID.randomUUID().toString();
        MDC.put("correlationId", cid);
        log.info("{} {}", req.getMethod(), req.getRequestURI());
        try { chain.doFilter(req, res); }
        finally {
```

```
log.info("status={}", res.getStatus());
MDC.clear();
}
}
}
```

7. OUTPUT

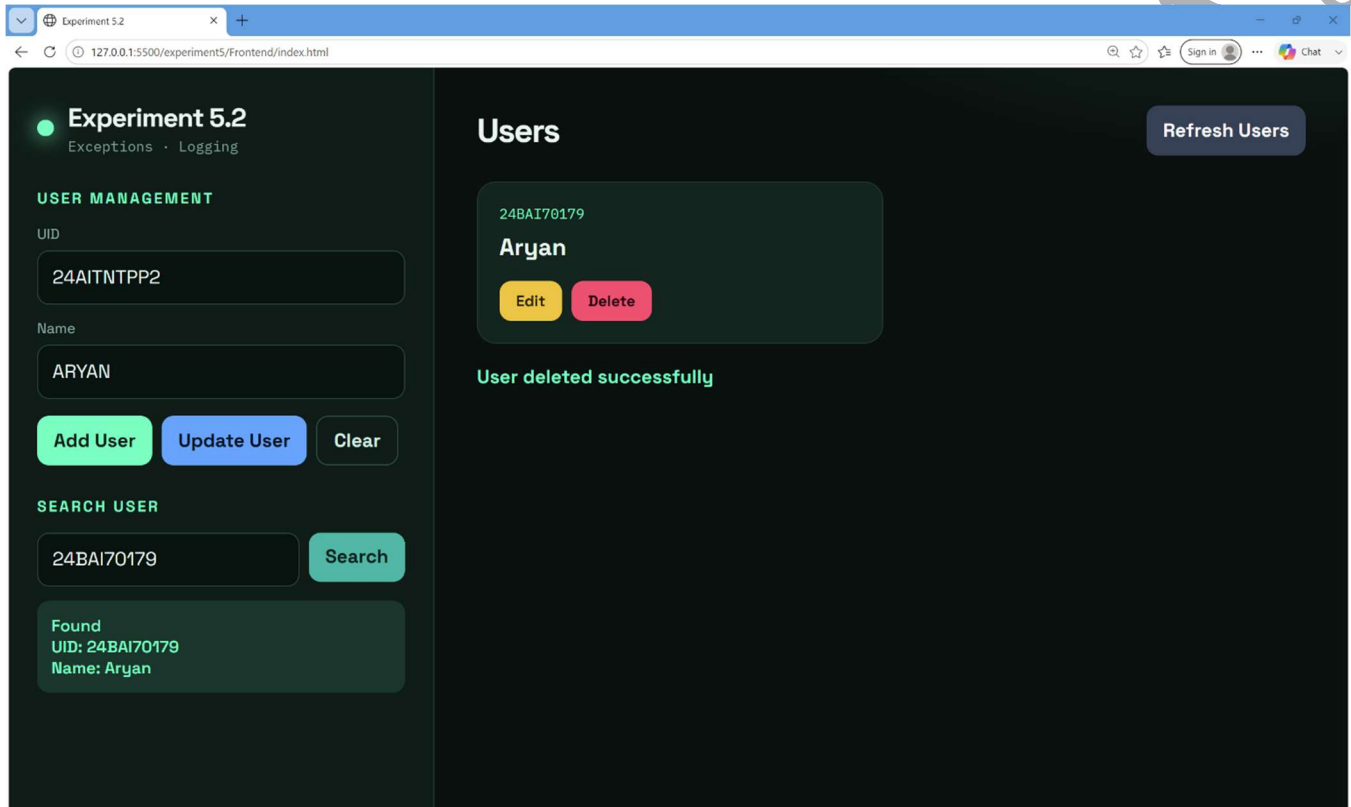


Figure 2: Output of Experiment 5.2

8. LEARNING OUTCOME

- Learned to centralize REST error handling with `@RestControllerAdvice`.
- Mapped domain and validation exceptions to correct HTTP status codes.
- Implemented request logging with a correlation identifier using a Filter and MDC.
- Understood why consistent error envelopes make frontend integration easier.